



# Slash Commands in Claude Code

A reference guide to turning repeated prompts into named, reusable workflows — `/generate-test-cases` and `/review` explained

## 🔗 WHAT'S INSIDE

1. The problem: ad-hoc instructions don't scale
2. What is a slash command? Built-in vs custom
3. Anatomy of a command — \$ARGUMENTS, \$1/\$2, frontmatter
4. Two ways to define one: commands vs Skills
5. Worked example: `/generate-test-cases`
6. Worked example: `/review` for automation code
7. CLAUDE.md vs Slash Command — division of responsibility
8. Built-in commands reference & self-test

# Ad-Hoc Instructions Don't Scale

Why a repeatable engineering task needs a name, not just a sentence

`CLAUDE.md` solves one problem: it keeps project-wide standards — test-case format, coverage rules, coding conventions — loaded into every session automatically, so nobody has to restate them. But it doesn't solve a second, related problem: the instruction you type to trigger a specific, repeatable task is still **free text**.

Take a project whose `CLAUDE.md` already defines the test-case format and coverage rules. Given an acceptance criterion like "A *registered user should be able to log in with a valid email and password*," asking Claude to "generate test cases for this story" works fine — Claude already knows the format from `CLAUDE.md`.

## ⚠ BUT THE INSTRUCTION ITSELF ISN'T STANDARDIZED

Free text drifts. One engineer says "generate test cases." Another says "create QA scenarios." A third says "generate positive and negative test cases." Claude can probably parse all of these — but for a workflow the whole team relies on, no one should have to remember or reinvent the exact wording each time.

## 🔑 The same task, phrased five different ways

"Generate test cases for this story."

"Create QA scenarios for this story."

"Generate positive and negative test cases."

"Write test scenarios, please."

### ✗ Not standardized

No shared wording — every engineer improvises their own phrasing

Claude can understand all of these — but a repeatable workflow shouldn't depend on everyone remembering how to invoke it.

## 🔑 THE FIX

Give the task a **name** — something like `/generate-test-cases` — so no one has to remember or reinvent the instruction. That named, reusable instruction is a **slash command**.

# What Exactly Is a Slash Command?

Definition, built-in vs custom, and how arguments work

## 📖 DEFINITION

A slash command is a **reusable instruction or workflow that you explicitly invoke by name** — e.g. `/generate-test-cases` or `/review`. Nothing happens until you type it.

## 🗃️ Two types of slash commands

### ⚙️ Built-in commands

Ship with Claude Code itself — session & project utilities such as `/init`, `/clear`, `/compact`, `/memory`, `/review`, `/agents`. The exact list varies by version — check `/help` in your installed CLI for the current set.

### ⚡ Custom commands

Commands **you define** for your own repeatable workflows — e.g. `/generate-test-cases` for turning a user story into test cases, or `/review` for checking code against project standards. These are the focus of this guide.

## 👤 Commands can accept input

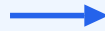
```
/generate-test-cases <user story>
```

A command isn't just a fixed phrase — it can take an **argument**, so the same named workflow runs against whatever story, file, or diff you pass it. Some commands accept **multiple** parameters, too (covered on the next page).

### Free text vs a named command

#### 😊 Free text

"Generate test cases for this story"  
"Create QA scenarios..."  
"Write positive/negative cases..."  
→ different every time, per person



#### ✅ A named command

```
/generate-test-cases  
  <paste story>
```

→ same instruction, every time, for everyone

# Anatomy of a Custom Command

Description, body, and \$ARGUMENTS — using a test-case command as the example

Custom commands are just **Markdown files**. The filename (minus `.md`) becomes the command name. Here's a command that generates test cases from a pasted user story:

```
Generate test cases for a pasted user story.

Read CLAUDE.md and follow the test-case format and coverage rules exactly.
$ARGUMENTS is the user story text.
Output only the test cases.
```

## Line 1 = Description

"Generate test cases for a pasted user story."

Shown next to the command in the slash menu

## Points back to CLAUDE.md

"Read CLAUDE.md and follow the format/coverage rules"

No rule duplication — one source of truth

## \$ARGUMENTS

"\$ARGUMENTS is the user story text."

Whatever you type after the command name

### 🔗 DESIGN PRINCIPLE

The test-case rules are **not copied** into the command file. They live in CLAUDE.md, which loads on every session start — the command just names the task and points back at the rules that matter for it. This keeps commands short and the rules in one place.

## 🔗 How \$ARGUMENTS works

`$ARGUMENTS` is **literal text substitution**: whatever you type after the command name gets dropped in wherever `$ARGUMENTS` appears in the command file.

```
You type: /generate-test-cases Title: User can log in with email and password. AC: 1. Valid credentials → dashboard...

Claude receives (after substitution):
Generate test cases for a pasted user story.
Read CLAUDE.md and follow the test-case format and coverage rules exactly.
Title: User can log in with email and password. AC: 1. Valid credentials → dashboard... is the user story text.
Output only the test cases.
```

### i BEYOND \$ARGUMENTS

Claude Code also supports `$1`, `$2` ... for **positional** arguments (e.g. an issue number plus a priority), inline **bash execution** with `!`git diff --cached`` to pull live shell output into the prompt, and `@path/to/file` to reference a file directly.

# Two Ways to Define a Command

A single instruction file, or a Skill folder

## Option 1 — Command file

A single Markdown file inside a `commands/` folder. Filename → command name.

```
$ mkdir -p .claude/commands
$ nano .claude/commands/generate-test-cases.md
```

## Option 2 — Skill

A **folder**, so it can carry supporting files alongside the instruction — and Claude can also load it automatically when relevant, not just by name.

```
$ mkdir -p .claude/skills/generate-test-cases
$ nano .claude/skills/generate-test-cases/SKILL.md
```

### WHICH ONE TO REACH FOR

`.claude/commands/` is the original, lightweight format. Anthropic now documents `.claude/skills/<name>/SKILL.md` as the current recommended format — same `/name` invocation, plus Claude can invoke it autonomously when it judges it relevant. The CLI supports both. Skills work best for larger, multi-file workflows; single-instruction commands are still a fine fit for simple, one-line tasks.

## Folder & scope reference

```
.claude/commands/           ← PROJECT-scoped, classic format
├─ generate-test-cases.md   ← invoked as /generate-test-cases
├─ review.md               ← invoked as /review
├─ frontend/
│   └─ component.md        ← invoked as /component (shows "project:frontend")

~/.claude/commands/         ← PERSONAL, all your projects, not shared
└─ security-review.md

.claude/skills/             ← RECOMMENDED current format
├─ generate-test-cases/
│   ├── SKILL.md           ← the instruction (+ frontmatter)
│   └─ examples/           ← optional supporting files, templates, etc.
├─ review/
│   └─ SKILL.md
```

## Optional frontmatter on a command

```
---
description: Fix a GitHub issue
argument-hint: [issue-number] [priority]
allowed-tools: Read, Grep, Glob
---
Fix issue #${1} with priority ${2}. Check the issue description and implement the fix.
```

### RULE OF THUMB

If a name conflict exists between a personal and a project command, only **one** is available — keep project-shared and personal command names distinct.

## /generate-test-cases in Practice

From acceptance criteria to formatted test cases, in one named step



### 📄 Sample acceptance criteria

```
Title: User can log in with email and password.  
Acceptance Criteria:  
1. Valid credentials should take the user to the dashboard.  
2. Invalid password should display an error.  
3. After 5 failed attempts, the account should be locked for 15 minutes.
```

### ▶ The invocation

```
/generate-test-cases Title: User can log in with email and password. Acceptance Criteria: 1. Valid credentials... 2.  
Invalid password... 3. After 5 failed attempts...
```

#### 🔑 NET RESULT

A repeated prompt becomes a **named, reusable workflow** — run by name, with the story passed in as an argument. Same format the project defines, hitting the same coverage, every single time.

## /review for Automation Code

A bigger instruction, and a command that flexes with how you call it

Where a test-case command can be a single line, `/review` packages a **bigger, multi-step instruction** — checking a Playwright test against a project's automation standards can easily run to several steps. That's exactly the kind of thing not worth retyping with every request.

### ✂ Three ways to invoke the same command



#### One specific file

```
/review tests/e2e-purchase.spec.ts
```



#### Whatever changed

```
/review (no argument → reviews the current diff)
```



#### Part of a pipeline

Wired into a Git pre-push hook or CI, so changed code is reviewed before it reaches the shared branch

#### 🔗 WHAT MAKES IT A "WORKFLOW," NOT JUST A SAVED SENTENCE

Connecting `/review` to a pre-push hook or CI pipeline means review happens **consistently**, without anyone having to remember to run it manually.

### 🔗 CLAUDE.md sets the standard — /review checks it

#### CLAUDE.md

"We use the Page Object Model."  
Tells Claude what GOOD looks like



#### /review

"Does this test follow it?"  
Checks the code AGAINST that standard

### 📝 A scaling consideration

As a project grows, a Test Case Generator's CLAUDE.md holds test-case rules, an automation project's CLAUDE.md holds coding standards, and commands like `/review` keep getting longer and more specialised. Loading every piece of that into context on every message isn't efficient — which is exactly the gap **Skills** are designed to close.

#### 🔗 RELATED CONCEPT — SKILLS

Skills move larger, reusable workflows (like test-case rules or a review checklist) into a folder-based format that Claude can load only when relevant, rather than keeping everything resident in context. They're the current recommended approach for anything beyond a short, single-purpose command.

# CLAUDE.md vs Slash Command

Same project, two very different responsibilities

|               | CLAUDE.MD                                                                | SLASH COMMAND                                                                     |
|---------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| When it loads | <b>Always on</b> — injected into every message whether you ask or not    | <b>On demand only</b> — nothing happens until you type it                         |
| What it holds | Standards that should be true no matter what you're doing in the project | One specific, repeatable job — stays thin, points back at CLAUDE.md for the rules |
| Example       | "We use the Page Object Model."                                          | <code>/review</code> — checks whether code follows it                             |
| Analogy       | The rulebook                                                             | The named play you call from the rulebook                                         |

## ⚠ WHY NOT JUST PUT THIS IN CLAUDE.MD?

Because the two have different responsibilities. If a command's full instructions lived in CLAUDE.md instead, that content would load into **every single message** — even when that task isn't what you're doing. Commands keep the always-on file lean, and only pull in the bigger instruction when you actually invoke it.

## 🧠 Mental model



### CLAUDE.md

What good looks like — standing rules, always loaded



### Slash Command

A named job you call — points back at CLAUDE.md, stays thin



### Skill

A folder-based workflow — loads only when relevant

## ✓ Self-test

### Q1

Why doesn't "Generate test cases for this story" scale as a team workflow?

### Q2

What does \$ARGUMENTS actually do to the command file's text?

### Q3

Name the two places a custom command's instructions can live.

### Q4

Why shouldn't Page Object Model rules go inside the `/review` command itself?